



Advanced Encryption Schemes

Cecilia Boschini and Kristina Hostáková

Outline

Recap Lecture 2

Naor–Yung: the idea

Non-Interactive Zero-Knowledge Proofs

Naor–Yung: IND-CCA1 security
Hybrid Argument Interlude

Public Key Encryption

A PKE is a triple of PPT algorithms $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ satisfying

- Correctness: Honestly encrypted messages correctly decrypt.
- Security: “Ciphertext leaks nothing about the plaintext”
- So far:
 - Semantic Security and IND-CPA (passive adversary)
 - IND-CCA1 and IND-CCA2 (stronger notions)

Three security notions for PKE

Same game (guess b from $ct_b \leftarrow \text{Enc}(pk, m_b)$); they differ only in the access to $\mathcal{O}_{\text{Dec}}(\cdot) = \text{Dec}(sk, \cdot)$:

	$\mathcal{O}_{\text{Dec}}$ before ct_b	$\mathcal{O}_{\text{Dec}}$ after ct_b	Adversary
IND-CPA	×	×	passive
IND-CCA1	✓	×	lunchtime (Naor–Yung '90)
IND-CCA2	✓	✓ $ct \neq ct_b$	active (Rackoff–Simon '91)

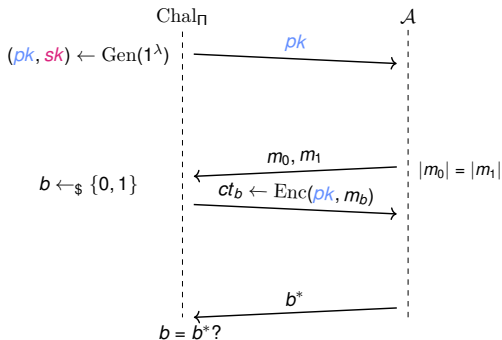
Three security notions for PKE

Same game (guess b from $ct_b \leftarrow \text{Enc}(pk, m_b)$); they differ only in the access to $\mathcal{O}_{\text{Dec}}(\cdot) = \text{Dec}(sk, \cdot)$:

	$\mathcal{O}_{\text{Dec}}$ before ct_b	$\mathcal{O}_{\text{Dec}}$ after ct_b	Adversary
IND-CPA	×	×	passive
IND-CCA1	✓	×	lunchtime (Naor–Yung '90)
IND-CCA2	✓	✓ $ct \neq ct_b$	active (Rackoff–Simon '91)

IND-CCA2 $\longrightarrow$ **IND-CCA1** $\longrightarrow$ **IND-CPA**

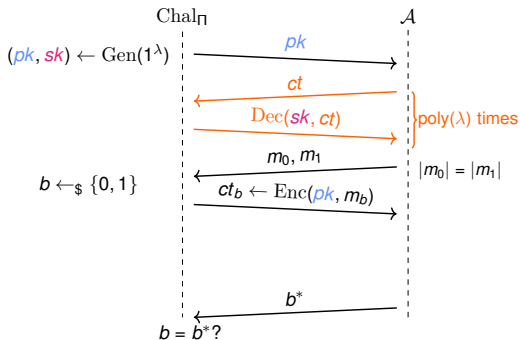
The three security experiments



Legend

- black: IND-CPA

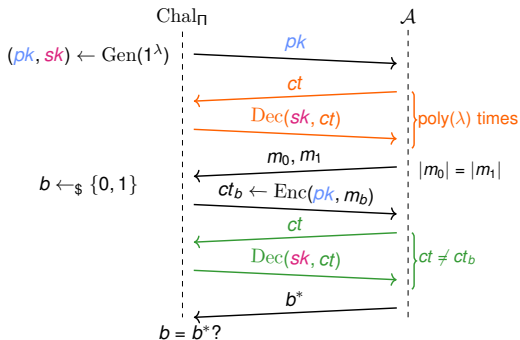
The three security experiments



Legend

- black: IND-CPA
- +orange: IND-CCA1

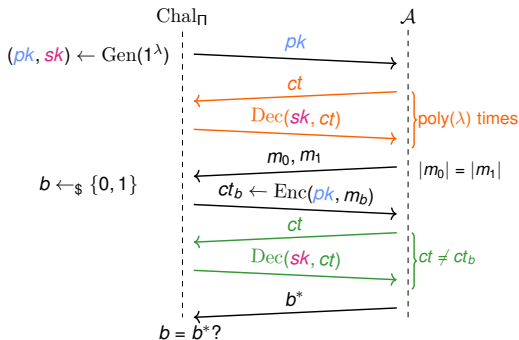
The three security experiments



Legend

- black: IND-CPA
- +orange: IND-CCA1
- +green: IND-CCA2

The three security experiments



Legend

- black: IND-CPA
- +orange: IND-CCA1
- +green: IND-CCA2

Π is **IND- X -secure** ($X \in \{\text{CPA}, \text{CCA1}, \text{CCA2}\}$) if $\forall$ PPT $\mathcal{A}$:

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-}X}(\lambda) := 2 \left| \frac{1}{2} - \Pr[\mathcal{A} \text{ wins}] \right| \leq \text{negl}(\lambda)$$

Our dream goal

$$\boxed{\Pi \text{ IND-CPA}} + \boxed{?} \Rightarrow \boxed{\Pi' \text{ IND-CCA2}}$$

Our dream goal

$$\boxed{\Pi \text{ IND-CPA}} + \boxed{?} \Rightarrow \boxed{\Pi' \text{ IND-CCA2}}$$

A *generic transformation*: from *any* IND-CPA scheme Π plus some extra machinery, build an IND-CCA2 scheme Π' – with a *proof*.

Our dream goal

$$\boxed{\Pi \text{ IND-CPA}} + \boxed{?} \Rightarrow \boxed{\Pi' \text{ IND-CCA2}}$$

A *generic transformation*: from *any* IND-CPA scheme Π plus some extra machinery, build an IND-CCA2 scheme Π' – with a *proof*.

- **Why generic?** IND-CPA schemes are plentiful (ElGamal, lattices, ...) and we reuse their proofs.
- **The question:** what is the “?”, and what does it cost?

Our dream goal

$$\boxed{\Pi \text{ IND-CPA}} + \boxed{?} \Rightarrow \boxed{\Pi' \text{ IND-CCA2}}$$

A *generic transformation*: from *any* IND-CPA scheme Π plus some extra machinery, build an IND-CCA2 scheme Π' – with a *proof*.

- **Why generic?** IND-CPA schemes are plentiful (ElGamal, lattices, ...) and we reuse their proofs.
- **The question:** what is the “?”, and what does it cost?

<i>Naor–Yung</i>	? = zero-knowledge proofs	this lecture
<i>Cramer–Shoup</i>	? = universal hash proofs	
<i>Fujisaki–Okamoto</i>	? = random oracle	

Outline

Recap Lecture 2

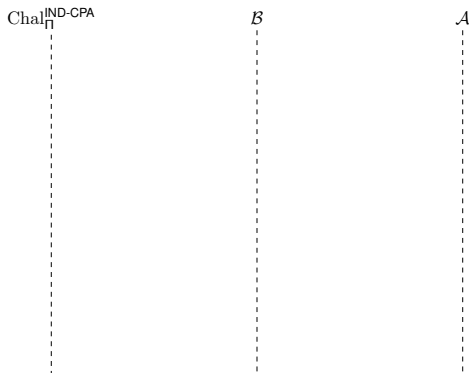
Naor–Yung: the idea

Non-Interactive Zero-Knowledge Proofs

Naor–Yung: IND-CCA1 security
Hybrid Argument Interlude

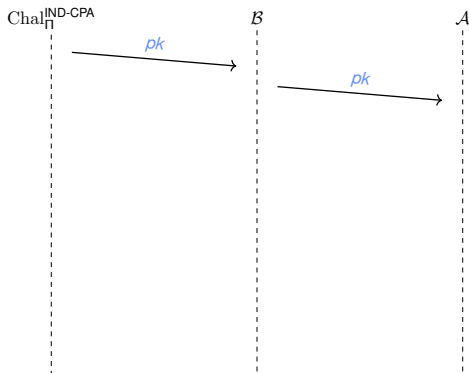
Recall: why this is hard

Naive attempt: Π IND-CPA scheme $\Rightarrow \Pi$ itself is IND-CCA1 secure?



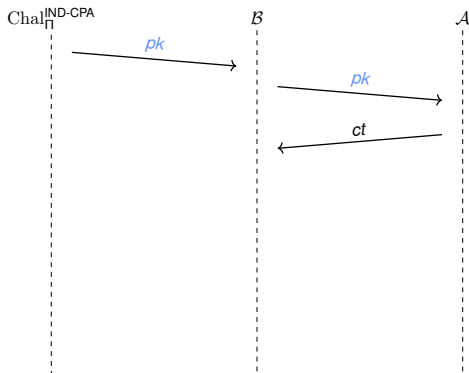
Recall: why this is hard

Naive attempt: Π IND-CPA scheme $\Rightarrow \Pi$ itself is IND-CCA1 secure?



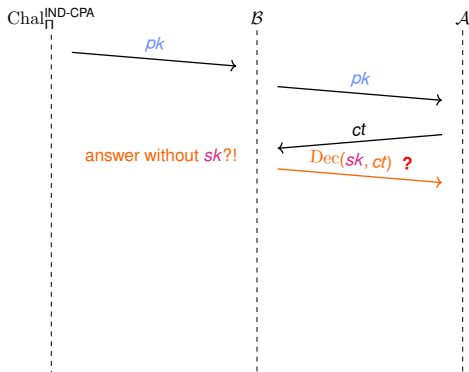
Recall: why this is hard

Naive attempt: Π IND-CPA scheme $\Rightarrow \Pi$ itself is IND-CCA1 secure?



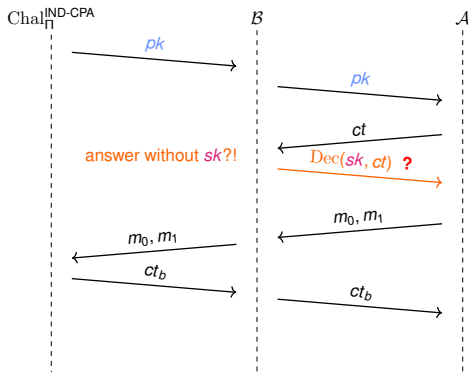
Recall: why this is hard

Naive attempt: Π IND-CPA scheme $\Rightarrow \Pi$ itself is IND-CCA1 secure?



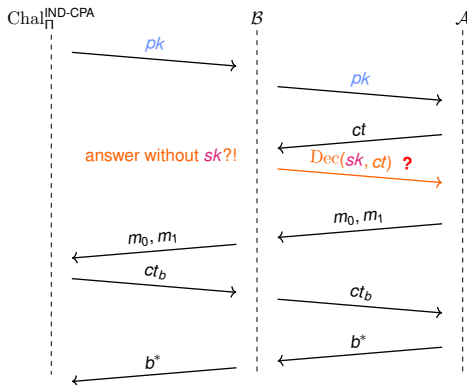
Recall: why this is hard

Naive attempt: Π IND-CPA scheme $\Rightarrow \Pi$ itself is IND-CCA1 secure?



Recall: why this is hard

Naive attempt: Π IND-CPA scheme $\Rightarrow \Pi$ itself is IND-CCA1 secure?



Core problem: $\mathcal{B}$ must *answer decryption queries without sk* .

Idea of Naor-Yung: encrypt twice

Let $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ be IND-CPA secure. Define Π_2 :

$\Pi_2 = (\Pi_2.\text{Gen}, \Pi_2.\text{Enc}, \Pi_2.\text{Dec})$

$\Pi_2.\text{Gen}(1^\lambda)$: $(pk_0, sk_0) \leftarrow \text{Gen}(1^\lambda), (pk_1, sk_1) \leftarrow \text{Gen}(1^\lambda)$
output $pk := (pk_0, pk_1), sk := sk_0$ (sk_1 thrown away)

Idea of Naor-Yung: encrypt twice

Let $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ be IND-CPA secure. Define Π_2 :

$$\Pi_2 = (\Pi_2.\text{Gen}, \Pi_2.\text{Enc}, \Pi_2.\text{Dec})$$

$\Pi_2.\text{Gen}(1^\lambda)$: $(pk_0, sk_0) \leftarrow \text{Gen}(1^\lambda), (pk_1, sk_1) \leftarrow \text{Gen}(1^\lambda)$
output $pk := (pk_0, pk_1), sk := sk_0$ (sk_1 thrown away)

$\Pi_2.\text{Enc}(pk, m)$: $c_0 \leftarrow \text{Enc}(pk_0, m), c_1 \leftarrow \text{Enc}(pk_1, m)$
output $ct := (c_0, c_1)$

Idea of Naor-Yung: encrypt twice

Let $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ be IND-CPA secure. Define Π_2 :

$\Pi_2 = (\Pi_2.\text{Gen}, \Pi_2.\text{Enc}, \Pi_2.\text{Dec})$

$\Pi_2.\text{Gen}(1^\lambda)$: $(pk_0, sk_0) \leftarrow \text{Gen}(1^\lambda), (pk_1, sk_1) \leftarrow \text{Gen}(1^\lambda)$
 output $pk := (pk_0, pk_1), sk := sk_0$ (sk_1 thrown away)

$\Pi_2.\text{Enc}(pk, m)$: $c_0 \leftarrow \text{Enc}(pk_0, m), c_1 \leftarrow \text{Enc}(pk_1, m)$
 output $ct := (c_0, c_1)$

$\Pi_2.\text{Dec}(sk, ct)$: parse $ct = (c_0, c_1)$, output $\text{Dec}(sk_0, c_0)$

Idea of Naor-Yung: encrypt twice

Let $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ be IND-CPA secure. Define Π_2 :

$$\Pi_2 = (\Pi_2.\text{Gen}, \Pi_2.\text{Enc}, \Pi_2.\text{Dec})$$

$\Pi_2.\text{Gen}(1^\lambda)$: $(pk_0, sk_0) \leftarrow \text{Gen}(1^\lambda), (pk_1, sk_1) \leftarrow \text{Gen}(1^\lambda)$
 output $pk := (pk_0, pk_1), sk := sk_0$ (sk_1 thrown away)

$\Pi_2.\text{Enc}(pk, m)$: $c_0 \leftarrow \text{Enc}(pk_0, m), c_1 \leftarrow \text{Enc}(pk_1, m)$
 output $ct := (c_0, c_1)$

$\Pi_2.\text{Dec}(sk, ct)$: parse $ct = (c_0, c_1)$, output $\text{Dec}(sk_0, c_0)$

Both keys decrypt honest ciphertexts:

$$\text{Dec}(sk_0, c_0) = \text{Dec}(sk_1, c_1) = m$$

How the reduction could work

Chal _{Π} ^{IND-CPA}

$\mathcal{B}$

$\mathcal{A}$

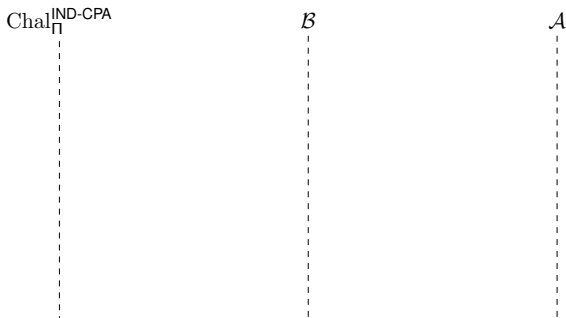
.....

.....

.....

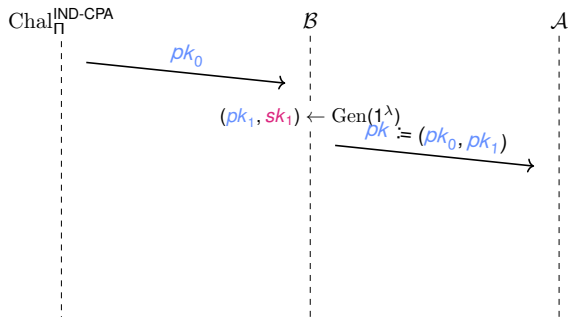
How the reduction could work

Reduce IND-CCA1 of Π_2 to IND-CPA of Π under key pk_0 :



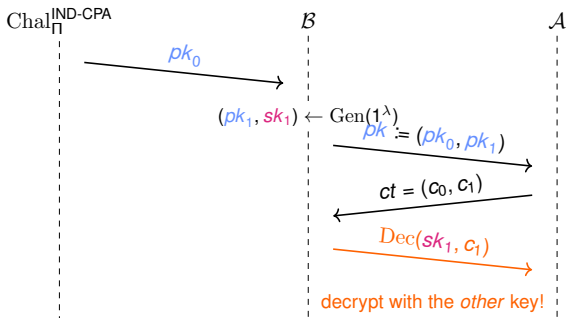
How the reduction could work

Reduce IND-CCA1 of Π_2 to IND-CPA of Π under key pk_0 :



How the reduction could work

Reduce IND-CCA1 of Π_2 to IND-CPA of Π under key pk_0 :



For an honest $ct = (\text{Enc}(pk_0, m), \text{Enc}(pk_1, m))$ the answer $\text{Dec}(sk_1, c_1) = m$ is **exactly right**.

(The challenge phase needs another trick — later.)

Problem: inconsistent ciphertexts

$\mathcal{A}$ is not obliged to send honest ciphertexts.

$\mathcal{A}$ tests the decryption oracle

- Compute $c_0 \leftarrow \text{Enc}(pk_0, m)$ and $c_1 \leftarrow \text{Enc}(pk_1, m')$ with $m \neq m'$
- Query $\mathcal{O}_{\text{Dec}}(ct := (c_0, c_1))$
- Real experiment answers $\text{Dec}(sk_0, c_0) = m$; $\mathcal{B}$ answers $\text{Dec}(sk_1, c_1) = m'$

Problem: inconsistent ciphertexts

$\mathcal{A}$ is not obliged to send honest ciphertexts.

$\mathcal{A}$ tests the decryption oracle

- Compute $c_0 \leftarrow \text{Enc}(pk_0, m)$ and $c_1 \leftarrow \text{Enc}(pk_1, m')$ with $m \neq m'$
- Query $\mathcal{O}_{\text{Dec}}(ct := (c_0, c_1))$
- Real experiment answers $\text{Dec}(sk_0, c_0) = m$; $\mathcal{B}$ answers $\text{Dec}(sk_1, c_1) = m'$

- $\mathcal{A}$ can *tell the simulation apart* from the real experiment
 - $\mathcal{A}$ may then behave arbitrarily
 - $\mathcal{B}$'s success is no longer related to $\mathcal{A}$'s

Problem: inconsistent ciphertexts

$\mathcal{A}$ is not obliged to send honest ciphertexts.

$\mathcal{A}$ tests the decryption oracle

- Compute $c_0 \leftarrow \text{Enc}(pk_0, m)$ and $c_1 \leftarrow \text{Enc}(pk_1, m')$ with $m \neq m'$
- Query $\mathcal{O}_{\text{Dec}}(ct := (c_0, c_1))$
- Real experiment answers $\text{Dec}(sk_0, c_0) = m$; $\mathcal{B}$ answers $\text{Dec}(sk_1, c_1) = m'$

- $\mathcal{A}$ can *tell the simulation apart* from the real experiment
 - $\mathcal{A}$ may then behave arbitrarily
 - $\mathcal{B}$'s success is no longer related to $\mathcal{A}$'s

Root cause: $\mathcal{B}$ cannot recognise that (c_0, c_1) is *inconsistent*.

What would fix it?

We want Dec to *reject* inconsistent $ct := (c_0, c_1)$. What must the consistency check satisfy?

requirement	why
-------------	-----

What would fix it?

We want Dec to *reject* inconsistent $ct := (c_0, c_1)$. What must the consistency check satisfy?

requirement	why
<i>publicly checkable</i>	$\mathcal{B}$ has only one key

What would fix it?

We want Dec to *reject* inconsistent $ct := (c_0, c_1)$. What must the consistency check satisfy?

requirement	why
<i>publicly checkable</i>	$\mathcal{B}$ has only one key
<i>non-leaking</i>	must not reveal m

What would fix it?

We want Dec to *reject* inconsistent $ct := (c_0, c_1)$. What must the consistency check satisfy?

requirement	why
<i>publicly checkable</i>	$\mathcal{B}$ has only one key
<i>non-leaking</i>	must not reveal m
<i>unforgeable</i>	inconsistent ct rejected

What would fix it?

We want Dec to *reject* inconsistent $ct := (c_0, c_1)$. What must the consistency check satisfy?

requirement	why	property of the proof
<i>publicly checkable</i>	$\mathcal{B}$ has only one key	non-interactive , public
<i>non-leaking</i>	must not reveal m	zero-knowledge
<i>unforgeable</i>	inconsistent ct rejected	sound

$\Rightarrow$ attach to ct a **non-interactive zero-knowledge proof** π that

“ c_0 and c_1 encrypt the same message”.

The Naor–Yung ciphertext

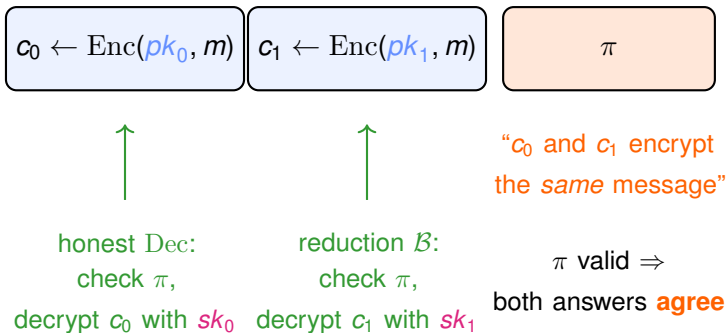
$$c_0 \leftarrow \text{Enc}(pk_0, m) \quad c_1 \leftarrow \text{Enc}(pk_1, m)$$

The Naor–Yung ciphertext

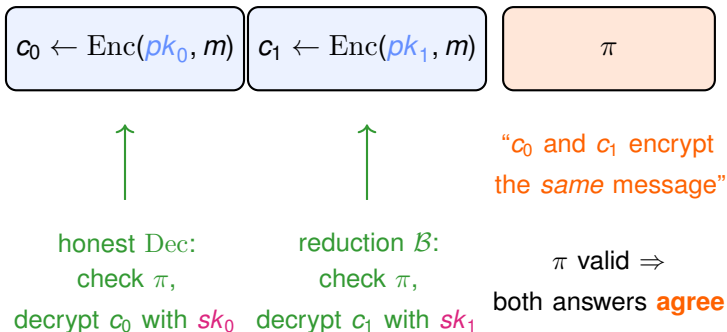
$$c_0 \leftarrow \text{Enc}(pk_0, m) \quad c_1 \leftarrow \text{Enc}(pk_1, m) \quad \pi$$

“ c_0 and c_1 encrypt
the *same* message”

The Naor–Yung ciphertext



The Naor–Yung ciphertext



Naor–Yung [NY90]: *encrypt twice and prove consistency*.

The tool we need first: **non-interactive zero-knowledge proofs**.

Outline

Recap Lecture 2

Naor–Yung: the idea

Non-Interactive Zero-Knowledge Proofs

Naor–Yung: IND-CCA1 security
Hybrid Argument Interlude

Languages, relations, witnesses

Definition 3.1: NP relation and language

A relation $\mathcal{R} \subseteq \{0, 1\}^* \times \{0, 1\}^*$ is an *NP relation* if $(x, w) \in \mathcal{R}$ is decidable in time $\text{poly}(|x|)$. Its language is

$$\mathcal{L}_{\mathcal{R}} := \{x : \exists w \text{ s.t. } (x, w) \in \mathcal{R}\}.$$

x is the *statement*, w a *witness* for $x \in \mathcal{L}_{\mathcal{R}}$.

Languages, relations, witnesses

Definition 3.1: NP relation and language

A relation $\mathcal{R} \subseteq \{0, 1\}^* \times \{0, 1\}^*$ is an *NP relation* if $(x, w) \in \mathcal{R}$ is decidable in time $\text{poly}(|x|)$. Its language is

$$\mathcal{L}_{\mathcal{R}} := \{x : \exists w \text{ s.t. } (x, w) \in \mathcal{R}\}.$$

x is the *statement*, w a *witness* for $x \in \mathcal{L}_{\mathcal{R}}$.

Example:

- $\mathbb{G}$ cyclic group of prime order p
- $\mathcal{R}_{\mathbb{G}, dlog} := \{(x := (g, h) \in \mathbb{G} \times \mathbb{G}, w \in \mathbb{Z}_p) \mid g^w = h\}$

Our language: consistency of two ciphertexts

$$\mathcal{R}_{\text{NY}} := \left\{ \left(\underbrace{(pk_0, pk_1, c_0, c_1)}_x, \underbrace{(m, r_0, r_1)}_w \right) : \right. \\ \left. c_0 = \text{Enc}(pk_0, m; r_0) \wedge c_1 = \text{Enc}(pk_1, m; r_1) \right\}$$

$$\mathcal{L}_{\text{NY}} = \{(pk_0, pk_1, c_0, c_1) : c_0, c_1 \text{ encrypt the same } m\}$$

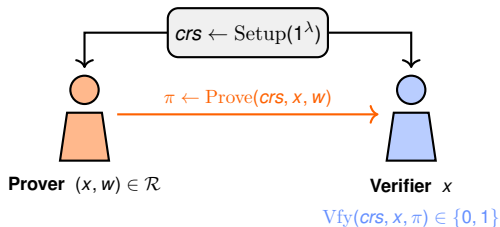
Our language: consistency of two ciphertexts

$$\mathcal{R}_{\text{NY}} := \left\{ \left(\underbrace{(\textcolor{blue}{pk}_0, \textcolor{blue}{pk}_1, c_0, c_1)}_x, \underbrace{(m, r_0, r_1)}_w \right) : \right. \\ \left. c_0 = \text{Enc}(\textcolor{blue}{pk}_0, m; r_0) \wedge c_1 = \text{Enc}(\textcolor{blue}{pk}_1, m; r_1) \right\}$$

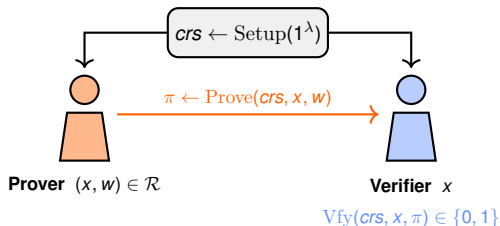
$$\mathcal{L}_{\text{NY}} = \{(\textcolor{blue}{pk}_0, \textcolor{blue}{pk}_1, c_0, c_1) : c_0, c_1 \text{ encrypt the same } m\}$$

- NP relation: checking $(x, w) \in \mathcal{R}_{\text{NY}}$ means running Enc twice.
- The encryptor knows the witness (m, r_0, r_1) — it just chose it.
- The decryptor / the reduction only knows x .

Non-interactive proofs in the CRS model



Non-interactive proofs in the CRS model



- The prover publishes π , anyone can verify.
- Both parties share a *common reference string* crs from a *trusted* setup. Without crs , NIZK exists only for trivial languages.

Non-interactive proofs: syntax

Definition 3.2: Non-interactive proof system for $\mathcal{R}$ – syntax

Three PPT algorithms:

- $\text{Setup}(1^\lambda)$: outputs a common reference string crs .
- $\text{Prove}(crs, x, w)$: outputs a proof π .
- $\text{Vfy}(crs, x, \pi)$: deterministic, outputs 1 (accept) or 0 (reject).

Non-interactive proofs: syntax

Definition 3.2: Non-interactive proof system for $\mathcal{R}$ – syntax

Three PPT algorithms:

- $\text{Setup}(1^\lambda)$: outputs a common reference string crs .
- $\text{Prove}(\text{crs}, x, w)$: outputs a proof π .
- $\text{Vfy}(\text{crs}, x, \pi)$: deterministic, outputs 1 (accept) or 0 (reject).

Three properties: *completeness*, *soundness*, *zero-knowledge*.

Completeness

Definition 3.3: (Perfect) completeness

For all λ , all $crs \in \text{Setup}(1^\lambda)$ and all $(x, w) \in \mathcal{R}$:

$$\Pr[\text{Vfy}(crs, x, \text{Prove}(crs, x, w)) = 1] = 1.$$

Completeness

Definition 3.3: (Perfect) completeness

For all λ , all $crs \in \text{Setup}(1^\lambda)$ and all $(x, w) \in \mathcal{R}$:

$$\Pr[\text{Vfy}(crs, x, \text{Prove}(crs, x, w)) = 1] = 1.$$

Honest provers with a valid witness always convince the verifier.

In Naor–Yung: honest ciphertexts are never rejected $\Rightarrow$ *correctness*.

Soundness

Definition 3.4: (Adaptive, computational) soundness

A proof system is *sound* if for all PPT $\mathcal{A}$

$$\text{Adv}_{\text{NIZK}, \mathcal{A}}^{\text{sound}}(\lambda) := \Pr \left[\begin{array}{l} \text{Vfy}(\text{crs}, x, \pi) = 1 \\ \wedge x \notin \mathcal{L}_{\mathcal{R}} \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right]$$

is negligible in λ .

Soundness

Definition 3.4: (Adaptive, computational) soundness

A proof system is *sound* if for all PPT $\mathcal{A}$

$$\text{Adv}_{\text{NIZK}, \mathcal{A}}^{\text{sound}}(\lambda) := \Pr \left[\begin{array}{l} \text{Vfy}(\text{crs}, x, \pi) = 1 \\ \wedge x \notin \mathcal{L}_{\mathcal{R}} \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right]$$

is negligible in λ .

- *Adaptive*: $\mathcal{A}$ picks the false statement *after* seeing crs .
In NY, $\mathcal{A}$ sees pk (and hence crs) first.
- *Computational*: only PPT provers are bounded (a.k.a. an *argument*). Fine for us.

Zero-knowledge: intuition

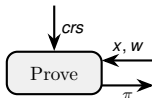
π should reveal nothing beyond “ $x \in \mathcal{L}_{\mathcal{R}}$ ”.

Zero-knowledge: intuition

π should reveal nothing beyond " $x \in \mathcal{L}_{\mathcal{R}}$ ".

- Formalisation: *real/ideal paradigm*

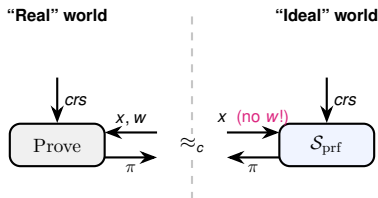
"Real" world



Zero-knowledge: intuition

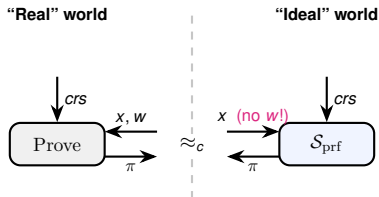
π should reveal nothing beyond " $x \in \mathcal{L}_{\mathcal{R}}$ ".

- Formalisation: *real/ideal paradigm*
- There is a PPT *simulator* $\mathcal{S}$ that "can fake proofs"
 - produces proofs *without any witness*
 - proof indistinguishable from "real" proofs
 - hence "real" proof cannot leak (much) information



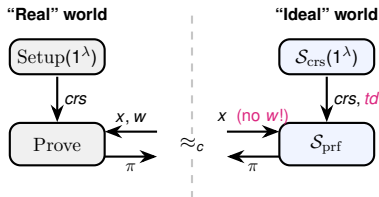
Zero-knowledge: intuition (2)

- How can $\mathcal{S}$ fake proofs without breaking *soundness*?



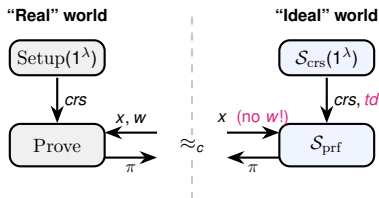
Zero-knowledge: intuition (2)

- How can $\mathcal{S}$ fake proofs without breaking *soundness*?
- Solution: $\mathcal{S}$ generates the *crs* itself, together with a *trapdoor* td .



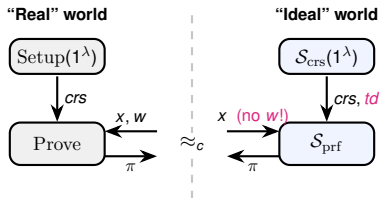
Zero-knowledge: intuition (2)

- How can $\mathcal{S}$ fake proofs without breaking *soundness*?
- Solution: $\mathcal{S}$ generates the *crs* itself, together with a *trapdoor* td .
- Real world: no td – so nobody can fake proofs there.



Zero-knowledge: intuition (3)

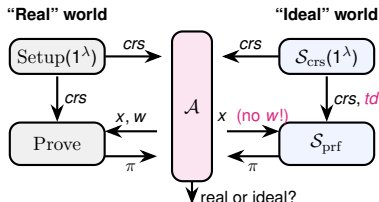
“Indistinguishable” for whom? For every PPT *distinguisher* $\mathcal{A}$, which



Zero-knowledge: intuition (3)

“Indistinguishable” for whom? For every PPT *distinguisher* $\mathcal{A}$, which

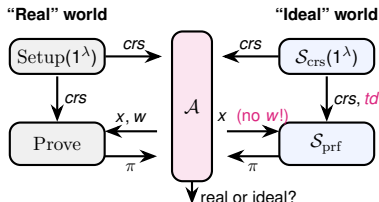
- receives crs — honest or simulated, it does not know which,



Zero-knowledge: intuition (3)

“Indistinguishable” for whom? For every PPT *distinguisher* $\mathcal{A}$, which

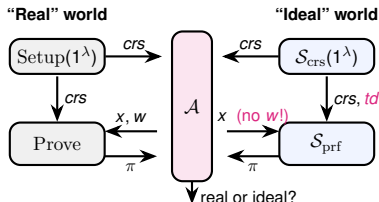
- receives crs — honest or simulated, it does not know which,
- chooses $(x, w) \in \mathcal{R}$ and receives a proof π for x ,



Zero-knowledge: intuition (3)

“Indistinguishable” for whom? For every PPT *distinguisher* $\mathcal{A}$, which

- receives crs — honest or simulated, it does not know which,
- chooses $(x, w) \in \mathcal{R}$ and receives a proof π for x ,
- guesses: real or ideal? ZK: $\mathcal{A}$ is right with probability $\approx 1/2$.



Zero-knowledge: definition

Definition 3.5: (Adaptive, single-theorem) zero-knowledge

A proof system is *zero-knowledge* if there is a PPT simulator $\mathcal{S} = (\mathcal{S}_{\text{crs}}, \mathcal{S}_{\text{prf}})$ s.t. for all PPT $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$

$$\text{Adv}_{\text{NIZK}, \mathcal{A}}^{\text{ZK}}(\lambda) := \left| \Pr[\text{ExpReal}_{\mathcal{A}}^{\text{ZK}}(\lambda) = 1] - \Pr[\text{ExpIdeal}_{\mathcal{A}}^{\text{ZK}}(\lambda) = 1] \right|$$

is negligible, where

$\text{ExpReal}_{\mathcal{A}}^{\text{ZK}}(\lambda):$
 $\text{crs} \leftarrow \text{Setup}(1^\lambda)$
 $(x, w, st) \leftarrow \mathcal{A}_0(\text{crs})$
 $\pi \leftarrow \text{Prove}(\text{crs}, x, w)$
 return $\mathcal{A}_1(st, \pi)$

$\text{ExpIdeal}_{\mathcal{A}}^{\text{ZK}}(\lambda):$
 $(\text{crs}, \text{td}) \leftarrow \mathcal{S}_{\text{crs}}(1^\lambda)$
 $(x, w, st) \leftarrow \mathcal{A}_0(\text{crs})$
 $\pi \leftarrow \mathcal{S}_{\text{prf}}(\text{crs}, \text{td}, x)$
 return $\mathcal{A}_1(st, \pi)$

(Both experiments output 0 if $(x, w) \notin \mathcal{R}$.)

Zero-knowledge: remarks

- $\mathcal{S}_{\text{crs}}$ must also produce a *crs* that *looks like* an honest one
Otherwise distinguishing trivial.

Zero-knowledge: remarks

- $\mathcal{S}_{\text{crs}}$ must also produce a *crs* that *looks like* an honest one
Otherwise distinguishing trivial.
- *Adaptive ZK*: $\mathcal{A}$ chooses the statement *after* seeing *crs*
In NY adversary gets *pk* first.

Zero-knowledge: remarks

- $\mathcal{S}_{\text{crs}}$ must also produce a *crs* that *looks like* an honest one
Otherwise distinguishing trivial.
- *Adaptive ZK*: $\mathcal{A}$ chooses the statement *after* seeing *crs*
In NY adversary gets *pk* first.
- *Single-theorem*: one simulated proof
Enough for NY – there is one challenge ciphertext.

Zero-knowledge: remarks

- $\mathcal{S}_{\text{crs}}$ must also produce a *crs* that *looks like* an honest one
Otherwise distinguishing trivial.
- *Adaptive ZK*: $\mathcal{A}$ chooses the statement *after* seeing *crs*
In NY adversary gets *pk* first.
- *Single-theorem*: one simulated proof
Enough for NY – there is one challenge ciphertext.
- Soundness and ZK pull in opposite directions:
 - honest *crs* $\Rightarrow$ nobody can fake proofs;
 - simulated *crs* $\Rightarrow \mathcal{S}$ can fake any proof

Where do NIZKs come from?

- NIZK for *all of NP* exists under standard assumptions:
trapdoor permutations [FLS90], pairings [GOS06], LWE [PS19], ...

Where do NIZKs come from?

- NIZK for *all of NP* exists under standard assumptions:
trapdoor permutations [FLS90], pairings [GOS06], LWE [PS19], ...
- In the ROM: Σ -protocols + Fiat–Shamir give efficient NIZKs for concrete languages
(e.g. “two ElGamal ciphertexts encrypt the same message”).

Where do NIZKs come from?

- NIZK for *all of NP* exists under standard assumptions:
trapdoor permutations [FLS90], pairings [GOS06], LWE [PS19], ...
- In the ROM: Σ -protocols + Fiat–Shamir give efficient NIZKs for concrete languages
(e.g. “two ElGamal ciphertexts encrypt the same message”).
- This course: *standard model*, NIZK as a black box with the three properties.
- Separate course of ZK proofs (by Jonathan Bootle, IBM Zurich)

Which properties does Naor–Yung need?

property	used for
----------	----------

completeness	honest ciphertexts decrypt	→ correctness
--------------	----------------------------	---------------

Which properties does Naor–Yung need?

property	used for	
completeness	honest ciphertexts decrypt	→ correctness
zero-knowledge	π leaks nothing on m	→ needed even for IND-CPA

Which properties does Naor–Yung need?

property	used for	
completeness	honest ciphertexts decrypt	→ correctness
zero-knowledge	π leaks nothing on m	→ needed even for IND-CPA
soundness	valid $\pi \Rightarrow (c_0, c_1)$ consistent	→ $\mathcal{B}$ decrypts with <i>either</i> key

Which properties does Naor–Yung need?

property	used for	
completeness	honest ciphertexts decrypt	→ correctness
zero-knowledge	π leaks nothing on m	→ needed even for IND-CPA
soundness	valid $\pi \Rightarrow (c_0, c_1)$ consistent	→ $\mathcal{B}$ decrypts with <i>either</i> key

Today: these properties suffices for *IND-CCA1*.

Next time: for *IND-CCA2* we need a *stronger* form of soundness.

Outline

Recap Lecture 2

Naor–Yung: the idea

Non-Interactive Zero-Knowledge Proofs

Naor–Yung: IND-CCA1 security
Hybrid Argument Interlude

The Naor–Yung construction

Building blocks: PKE $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$, randomness space $\mathcal{R}$ and message space $\mathcal{M}$; NIZK = (Setup, Prove, Vfy) for $\mathcal{R}_{\text{NY}}$.

$\Pi_{\text{NY}} = (\Pi_{\text{NY}}.\text{Gen}, \Pi_{\text{NY}}.\text{Enc}, \Pi_{\text{NY}}.\text{Dec})$, message space $\mathcal{M}$

$\Pi_{\text{NY}}.\text{Gen}(1^\lambda)$:

- $(pk_0, sk_0) \leftarrow \text{Gen}(1^\lambda), (pk_1, sk_1) \leftarrow \text{Gen}(1^\lambda), crs \leftarrow \text{Setup}(1^\lambda)$
- $pk := (pk_0, pk_1, crs), \quad sk := sk_0 \quad (sk_1 \text{ is erased})$

The Naor–Yung construction

Building blocks: PKE $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$, randomness space R and message space $\mathcal{M}$; NIZK = (Setup, Prove, Vfy) for $\mathcal{R}_{\text{NY}}$.

$\Pi_{\text{NY}} = (\Pi_{\text{NY}}.\text{Gen}, \Pi_{\text{NY}}.\text{Enc}, \Pi_{\text{NY}}.\text{Dec})$, message space $\mathcal{M}$

$\Pi_{\text{NY}}.\text{Gen}(1^\lambda)$:

- $(pk_0, sk_0) \leftarrow \text{Gen}(1^\lambda)$, $(pk_1, sk_1) \leftarrow \text{Gen}(1^\lambda)$, $crs \leftarrow \text{Setup}(1^\lambda)$
- $pk := (pk_0, pk_1, crs)$, $sk := sk_0$ (sk_1 is erased)

$\Pi_{\text{NY}}.\text{Enc}(pk, m)$:

- $r_0, r_1 \leftarrow_{\$} R$, $c_0 := \text{Enc}(pk_0, m; r_0)$, $c_1 := \text{Enc}(pk_1, m; r_1)$
- $\pi \leftarrow \text{Prove}(crs, x = (pk_0, pk_1, c_0, c_1), w := (m, r_0, r_1))$
- output $ct := (c_0, c_1, \pi)$

The Naor–Yung construction

Building blocks: PKE $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$, randomness space $\mathcal{R}$ and message space $\mathcal{M}$; NIZK = (Setup, Prove, Vfy) for $\mathcal{R}_{\text{NY}}$.

$\Pi_{\text{NY}} = (\Pi_{\text{NY}}.\text{Gen}, \Pi_{\text{NY}}.\text{Enc}, \Pi_{\text{NY}}.\text{Dec})$, message space $\mathcal{M}$

$\Pi_{\text{NY}}.\text{Gen}(1^\lambda)$:

- $(pk_0, sk_0) \leftarrow \text{Gen}(1^\lambda)$, $(pk_1, sk_1) \leftarrow \text{Gen}(1^\lambda)$, $crs \leftarrow \text{Setup}(1^\lambda)$
- $pk := (pk_0, pk_1, crs)$, $sk := sk_0$ (sk_1 is erased)

$\Pi_{\text{NY}}.\text{Enc}(pk, m)$:

- $r_0, r_1 \xleftarrow{\$} \mathcal{R}$, $c_0 := \text{Enc}(pk_0, m; r_0)$, $c_1 := \text{Enc}(pk_1, m; r_1)$
- $\pi \leftarrow \text{Prove}(crs, x = (pk_0, pk_1, c_0, c_1), w := (m, r_0, r_1))$
- output $ct := (c_0, c_1, \pi)$

$\Pi_{\text{NY}}.\text{Dec}(sk, ct)$:

- parse $ct = (c_0, c_1, \pi)$; if $\text{Vfy}(crs, (pk_0, pk_1, c_0, c_1), \pi) = 0$: output $\perp$
- else output $\text{Dec}(sk_0, c_0)$

Correctness

Theorem 3.1: Correctness of Naor–Yung

If Π is perfectly correct and NIZK is perfectly complete, then Π_{NY} is perfectly correct.

Correctness

Theorem 3.1: Correctness of Naor–Yung

If Π is perfectly correct and NIZK is perfectly complete, then Π_{NY} is perfectly correct.

Proof. Let $ct = (c_0, c_1, \pi) \leftarrow \Pi_{\text{NY}}.\text{Enc}(pk, m)$.

- $((pk_0, pk_1, c_0, c_1), (m, r_0, r_1)) \in \mathcal{R}_{\text{NY}}$ by construction
 $\Rightarrow \text{Vfy}$ accepts (completeness).
- $\text{Dec}(sk_0, c_0) = \text{Dec}(sk_0, \text{Enc}(pk_0, m; r_0)) = m$
 correctness of Π . □

Correctness

Theorem 3.1: Correctness of Naor–Yung

If Π is perfectly correct and NIZK is perfectly complete, then Π_{NY} is perfectly correct.

Proof. Let $ct = (c_0, c_1, \pi) \leftarrow \Pi_{\text{NY}}.\text{Enc}(pk, m)$.

- $((pk_0, pk_1, c_0, c_1), (m, r_0, r_1)) \in \mathcal{R}_{\text{NY}}$ by construction
 $\Rightarrow \text{Vfy}$ accepts (completeness).
- $\text{Dec}(sk_0, c_0) = \text{Dec}(sk_0, \text{Enc}(pk_0, m; r_0)) = m$
 correctness of Π . □

Cost: $|ct| = 2 \cdot |ct_{\Pi}| + |\pi|$, and one Vfy per decryption.

Security

Theorem 3.2: IND-CCA1 security of Naor–Yung [NY90]

If Π is IND-CPA secure and NIZK is complete, sound and zero-knowledge, then Π_{NY} is IND-CCA1 secure.

Security

Theorem 3.2: IND-CCA1 security of Naor–Yung [NY90]

If Π is IND-CPA secure and NIZK is complete, sound and zero-knowledge, then Π_{NY} is IND-CCA1 secure.

Proof strategy?

Security

Theorem 3.2: IND-CCA1 security of Naor–Yung [NY90]

If Π is IND-CPA secure and NIZK is complete, sound and zero-knowledge, then Π_{NY} is IND-CCA1 secure.

Proof strategy? By contrapositive?

Security

Theorem 3.2: IND-CCA1 security of Naor–Yung [NY90]

If Π is IND-CPA secure and NIZK is complete, sound and zero-knowledge, then Π_{NY} is IND-CCA1 secure.

Proof strategy? By contrapositive? We have many assumptions!

$\text{Assumption}_1 \wedge \text{Assumption}_2 \wedge \text{Assumption}_3 \Rightarrow \text{Statement}$

Hence we would be proving

$\neg \text{Statement} \Rightarrow \neg \text{Assumption}_1 \vee \neg \text{Assumption}_2 \vee \neg \text{Assumption}_3$

Security

Theorem 3.2: IND-CCA1 security of Naor–Yung [NY90]

If Π is IND-CPA secure and NIZK is complete, sound and zero-knowledge, then Π_{NY} is IND-CCA1 secure.

Proof strategy? By contrapositive? We have many assumptions!

$\text{Assumption}_1 \wedge \text{Assumption}_2 \wedge \text{Assumption}_3 \Rightarrow \text{Statement}$

Hence we would be proving

$\neg \text{Statement} \Rightarrow \neg \text{Assumption}_1 \vee \neg \text{Assumption}_2 \vee \neg \text{Assumption}_3$

Complicated!

Proof strategy

Direct proof.

- Fix an arbitrary PPT IND-CCA1 adversary $\mathcal{A}$ against Π_{NY} .
- Goal: show that $\text{Adv}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(\lambda)$ is negligible.

Proof strategy

Direct proof.

- Fix an arbitrary PPT IND-CCA1 adversary $\mathcal{A}$ against Π_{NY} .
- Goal: show that $\text{Adv}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(\lambda)$ is negligible.
- Lemma 3.3 (coming up):
 $\text{Adv}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}$ = how *differently* $\mathcal{A}$ behaves for $b = 0$ and $b = 1$.

Proof strategy

Direct proof.

- Fix an arbitrary PPT IND-CCA1 adversary $\mathcal{A}$ against Π_{NY} .
- Goal: show that $\text{Adv}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(\lambda)$ is negligible.
- Lemma 3.3 (coming up):
 $\text{Adv}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}$ = how *differently* $\mathcal{A}$ behaves for $b = 0$ and $b = 1$.

Basic idea: split the distance “ $b = 0 \leftrightarrow b = 1$ ” into small steps,
reduce to one assumption per step.

$\Rightarrow$ **hybrid argument** (a.k.a. game hopping)

Outline

Recap Lecture 2

Naor–Yung: the idea

Non-Interactive Zero-Knowledge Proofs

Naor–Yung: IND-CCA1 security
Hybrid Argument Interlude

The experiments $\text{Game}(0)$ and $\text{Game}(1)$

IND-CCA1 experiment with the challenge bit fixed:

$\text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(0)$

1. $(pk, sk) \leftarrow \text{Gen}(1^\lambda)$; run $\mathcal{A}(pk)$
2. $\mathcal{O}_{\text{Dec}}(ct)$: return $\text{Dec}(sk, ct)$
3. $\mathcal{A}$ outputs (m_0, m_1) , $|m_0| = |m_1|$
4. $ct^* \leftarrow \text{Enc}(pk, m_0)$
5. $b^* \leftarrow \mathcal{A}(ct^*)$; output b^*

$\text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(1)$

1. $(pk, sk) \leftarrow \text{Gen}(1^\lambda)$; run $\mathcal{A}(pk)$
2. $\mathcal{O}_{\text{Dec}}(ct)$: return $\text{Dec}(sk, ct)$
3. $\mathcal{A}$ outputs (m_0, m_1) , $|m_0| = |m_1|$
4. $ct^* \leftarrow \text{Enc}(pk, m_1)$
5. $b^* \leftarrow \mathcal{A}(ct^*)$; output b^*

The experiments $\text{Game}(0)$ and $\text{Game}(1)$

IND-CCA1 experiment with the challenge bit fixed:

$\text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(0)$

1. $(pk, sk) \leftarrow \text{Gen}(1^\lambda)$; run $\mathcal{A}(pk)$
2. $\mathcal{O}_{\text{Dec}}(ct)$: return $\text{Dec}(sk, ct)$
3. $\mathcal{A}$ outputs (m_0, m_1) , $|m_0| = |m_1|$
4. $ct^* \leftarrow \text{Enc}(pk, m_0)$
5. $b^* \leftarrow \mathcal{A}(ct^*)$; output b^*

$\text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(1)$

1. $(pk, sk) \leftarrow \text{Gen}(1^\lambda)$; run $\mathcal{A}(pk)$
2. $\mathcal{O}_{\text{Dec}}(ct)$: return $\text{Dec}(sk, ct)$
3. $\mathcal{A}$ outputs (m_0, m_1) , $|m_0| = |m_1|$
4. $ct^* \leftarrow \text{Enc}(pk, m_1)$
5. $b^* \leftarrow \mathcal{A}(ct^*)$; output b^*

IND-CCA1 experiment for Π and $\mathcal{A}$

1. $b \leftarrow_{\$} \{0, 1\}$ and $b^* \leftarrow \text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(b)$
2. $\mathcal{A}$ wins iff $b^* = b$.

Interlude (1): advantage as a distance

Lemma 3.3: Advantage as a distance

For every PKE scheme Π and every PPT adversary $\mathcal{A}$:

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}} = |\Pr[\text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(1) = 1] - \Pr[\text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(0) = 1]|.$$

(The same holds for IND-CPA and IND-CCA2.)

Interlude (1): advantage as a distance

Lemma 3.3: Advantage as a distance

For every PKE scheme Π and every PPT adversary $\mathcal{A}$:

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}} = |\Pr[\text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(1) = 1] - \Pr[\text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(0) = 1]|.$$

(The same holds for IND-CPA and IND-CCA2.)

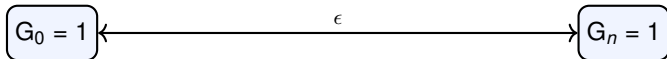
Proof. By hand

Interlude (2): advantage as a distance

- We have a fixed adversary $\mathcal{A}$
- A concrete scheme Π
- Let us denote:

$$G_0 := \text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(0), \quad G_n := \text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(1)$$

- Lemma 3.3 tells us we need to bound the distance between

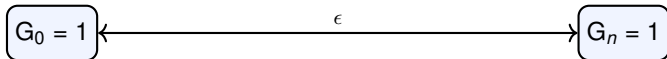


Interlude (2): advantage as a distance

- We have a fixed adversary $\mathcal{A}$
- A concrete scheme Π
- Let us denote:

$$G_0 := \text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(0), \quad G_n := \text{Game}_{\Pi, \mathcal{A}}^{\text{IND-CCA1}}(1)$$

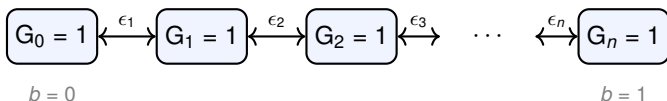
- Lemma 3.3 tells us we need to bound the distance between



If there are many assumption, none will bound this distance directly.

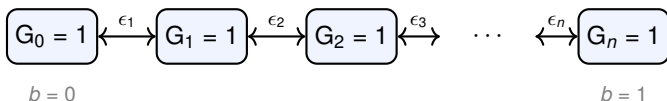
Interlude (3): the chain of games

- Insert *intermediate games* $G_1, \dots, G_{n-1}$
- Each differing from the previous one in *one small change*



Interlude (3): the chain of games

- Insert *intermediate games* $G_1, \dots, G_{n-1}$
- Each differing from the previous one in *one small change*

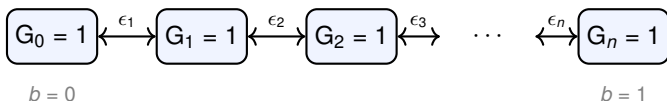


Triangle inequality, with $\epsilon_i := |\Pr[G_i = 1] - \Pr[G_{i-1} = 1]|$:

$$\text{Adv}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}} = |\Pr[G_n = 1] - \Pr[G_0 = 1]| \leq \sum_{i=1}^n \epsilon_i.$$

Interlude (3): the chain of games

- Insert *intermediate games* $G_1, \dots, G_{n-1}$
- Each differing from the previous one in *one small change*



Triangle inequality, with $\epsilon_i := |\Pr[G_i = 1] - \Pr[G_{i-1} = 1]|$:

$$\text{Adv}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}} = |\Pr[G_n = 1] - \Pr[G_0 = 1]| \leq \sum_{i=1}^n \epsilon_i.$$

If $n = \text{poly}(\lambda)$ and each $\epsilon_i \leq \text{negl} \Rightarrow \text{Adv} \leq \text{negl}$.

Interlude (4): bounding one hop

Two standard ways to bound ϵ_i :

Interlude (4): bounding one hop

Two standard ways to bound ϵ_i :

- 1 *Reduction hop.* G_{i-1} and G_i are the two “sides” of an assumption’s experiment.

- $\mathcal{B}$ receives the assumption’s challenge
- $\mathcal{B}$ runs $\mathcal{A}$, and embeds the challenge so that
side 0 $\Rightarrow \mathcal{A}$ sees G_{i-1} , side 1 $\Rightarrow \mathcal{A}$ sees G_i .
- $\mathcal{B}$ outputs $\mathcal{A}$ ’s output

$\Rightarrow \epsilon_i \leq \text{Adv}_{\mathcal{B}}$. (ElGamal proof: one hop, reduction to DDH.)

Interlude (4): bounding one hop

Two standard ways to bound ϵ_i :

- 1 **Reduction hop.** G_{i-1} and G_i are the two “sides” of an assumption’s experiment.

- $\mathcal{B}$ receives the assumption’s challenge
- $\mathcal{B}$ runs $\mathcal{A}$, and embeds the challenge so that
side 0 $\Rightarrow \mathcal{A}$ sees G_{i-1} , side 1 $\Rightarrow \mathcal{A}$ sees G_i .
- $\mathcal{B}$ outputs $\mathcal{A}$ ’s output

$\Rightarrow \epsilon_i \leq \text{Adv}_{\mathcal{B}}$. (ElGamal proof: one hop, reduction to DDH.)

- 2 **Bad-event hop.** G_{i-1} and G_i proceed *identically* unless an event Bad occurs.

- Then $\epsilon_i \leq \text{Pr}[\text{Bad}]$
- We bound $\text{Pr}[\text{Bad}]$ separately

Outline

Recap Lecture 2

Naor–Yung: the idea

Non-Interactive Zero-Knowledge Proofs

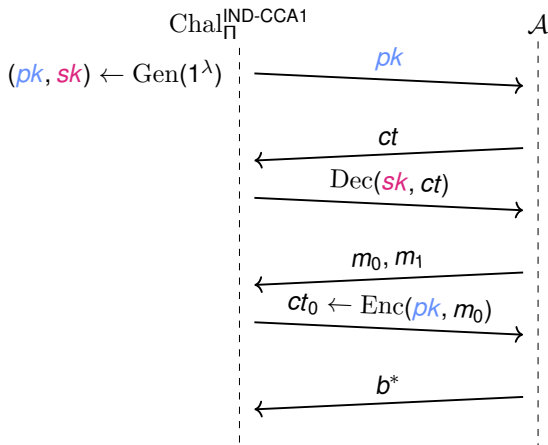
Naor–Yung: IND-CCA1 security
Hybrid Argument Interlude

Back to our proof

- Fix arbitrary PPT $\mathcal{A}$ playing IND-CCA1 against Π_{NY}
- We need to show

$$\left| \Pr[\text{Game}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(0) = 1] - \Pr[\text{Game}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(1) = 1] \right| \leq \text{negl}$$

The game $G_0 = \text{Game}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(0)$



The game $G_0 = \text{Game}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(0)$

$\text{Game}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(0)$

1. $(pk_0, sk_0) \leftarrow \text{Gen}(1^\lambda)$, $(pk_1, sk_1) \leftarrow \text{Gen}(1^\lambda)$, $crs \leftarrow \text{Setup}(1^\lambda)$
2. $pk := (pk_0, pk_1, crs)$; run $\mathcal{A}(pk)$
3. on query $\mathcal{O}_{\text{Dec}}(c_0, c_1, \pi)$: if $\text{Vfy}(crs, (pk_0, pk_1, c_0, c_1), \pi) = 0$ return $\perp$, else return $\text{Dec}(sk_0, c_0)$
4. $\mathcal{A}$ outputs (m_0, m_1) with $|m_0| = |m_1|$
5. $r_0, r_1 \xleftarrow{\$} \mathcal{R}$, $c_0^* := \text{Enc}(pk_0, m_0; r_0)$, $c_1^* := \text{Enc}(pk_1, m_0; r_1)$
6. $\pi^* \leftarrow \text{Prove}(crs, (pk_0, pk_1, c_0^*, c_1^*), (m_0, r_0, r_1))$
7. $b^* \leftarrow \mathcal{A}(ct^* = (c_0^*, c_1^*, \pi^*))$; output b^*

Proof of Theorem 3.2: the games

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—

Proof of Theorem 3.2: the games

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge

Proof of Theorem 3.2: the games

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge
G_2	m_0	m_1	simulated	sk_0	IND-CPA of pk_1

Proof of Theorem 3.2: the games

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge
G_2	m_0	m_1	simulated	sk_0	IND-CPA of pk_1
G_3	m_0	m_1	simulated	sk_1	soundness

Proof of Theorem 3.2: the games

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge
G_2	m_0	m_1	simulated	sk_0	IND-CPA of pk_1
G_3	m_0	m_1	simulated	sk_1	soundness
G_4	m_1	m_1	simulated	sk_1	IND-CPA of pk_0

Proof of Theorem 3.2: the games

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge
G_2	m_0	m_1	simulated	sk_0	IND-CPA of pk_1
G_3	m_0	m_1	simulated	sk_1	soundness
G_4	m_1	m_1	simulated	sk_1	IND-CPA of pk_0
G_5	m_1	m_1	simulated	sk_0	soundness

Proof of Theorem 3.2: the games

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge
G_2	m_0	m_1	simulated	sk_0	IND-CPA of pk_1
G_3	m_0	m_1	simulated	sk_1	soundness
G_4	m_1	m_1	simulated	sk_1	IND-CPA of pk_0
G_5	m_1	m_1	simulated	sk_0	soundness
G_6	m_1	m_1	real	sk_0	zero-knowledge

Proof of Theorem 3.2: the games

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge
G_2	m_0	m_1	simulated	sk_0	IND-CPA of pk_1
G_3	m_0	m_1	simulated	sk_1	soundness
G_4	m_1	m_1	simulated	sk_1	IND-CPA of pk_0
G_5	m_1	m_1	simulated	sk_0	soundness
G_6	m_1	m_1	real	sk_0	zero-knowledge

“simulated”: $(crs, td) \leftarrow S_{crs}(1^\lambda)$, $\pi^* \leftarrow S_{prf}(crs, td, (pk_0, pk_1, c_0^*, c_1^*))$

so no witness needed. In G_2, G_3, G_4 the statement is **false**!

Proof of Theorem 3.2: where are we?

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge
G_2	m_0	m_1	simulated	sk_0	IND-CPA of pk_1
G_3	m_0	m_1	simulated	sk_1	soundness
G_4	m_1	m_1	simulated	sk_1	IND-CPA of pk_0
G_5	m_1	m_1	simulated	sk_0	soundness
G_6	m_1	m_1	real	sk_0	zero-knowledge

Hop $G_0 \rightarrow G_1$: zero-knowledge

Only change: crs, π^* simulated instead of real. Reduction to ZK.

Distinguisher $\mathcal{D} = (\mathcal{D}_0, \mathcal{D}_1)$ against ZK

$\mathcal{D}_0(crs)$:

- $(pk_0, sk_0), (pk_1, sk_1) \leftarrow \text{Gen}(1^\lambda)$; run $\mathcal{A}(pk_0, pk_1, crs)$
- answer $\mathcal{O}_{\text{Dec}}$ as $\Pi_{\text{NY}}.\text{Dec}$ with sk_0 ;
- on (m_0, m_1) : $c_i^* := \text{Enc}(pk_i, m_0; r_i)$, output $x := (pk_0, pk_1, c_0^*, c_1^*)$, $w := (m_0, r_0, r_1)$

$\mathcal{D}_1(st, \pi^*)$: give $ct^* = (c_0^*, c_1^*, \pi^*)$ to $\mathcal{A}$; output $\mathcal{A}$'s bit b^*

Hop $G_0 \rightarrow G_1$: zero-knowledge

Only change: crs, π^* simulated instead of real. Reduction to ZK.

Distinguisher $\mathcal{D} = (\mathcal{D}_0, \mathcal{D}_1)$ against ZK

$\mathcal{D}_0(crs)$:

- $(pk_0, sk_0), (pk_1, sk_1) \leftarrow \text{Gen}(1^\lambda)$; run $\mathcal{A}(pk_0, pk_1, crs)$
- answer $\mathcal{O}_{\text{Dec}}$ as $\Pi_{\text{NY}}.\text{Dec}$ with sk_0 ;
- on (m_0, m_1) : $c_i^* := \text{Enc}(pk_i, m_0; r_i)$, output $x := (pk_0, pk_1, c_0^*, c_1^*)$, $w := (m_0, r_0, r_1)$

$\mathcal{D}_1(st, \pi^*)$: give $ct^* = (c_0^*, c_1^*, \pi^*)$ to $\mathcal{A}$; output $\mathcal{A}$'s bit b^*

- $\mathcal{D}$ knows $sk_0 \Rightarrow$ can answer $\mathcal{O}_{\text{Dec}}$.
- $(x, w) \in \mathcal{R}_{\text{NY}}$, as both c_i^* encrypt m_0 .
- $\text{ExpReal}^{\text{ZK}}$: $\mathcal{A}$ sees G_0 . $\text{ExpIdeal}^{\text{ZK}}$: $\mathcal{A}$ sees G_1 .
 $\Rightarrow \epsilon_1 = \text{Adv}_{\text{NIZK}, \mathcal{D}}^{\text{ZK}}(\lambda)$.

Proof of Theorem 3.2: where are we?

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge
G_2	m_0	m_1	simulated	sk_0	IND-CPA of pk_1
G_3	m_0	m_1	simulated	sk_1	soundness
G_4	m_1	m_1	simulated	sk_1	IND-CPA of pk_0
G_5	m_1	m_1	simulated	sk_0	soundness
G_6	m_1	m_1	real	sk_0	zero-knowledge

Hop $G_1 \rightarrow G_2$: IND-CPA of pk_1

Only change: c_1^* encrypts m_1 instead of m_0 . Reduction to IND-CPA.

$\mathcal{B}$ against IND-CPA of Π

- Receive pk_1 ; $(pk_0, sk_0) \leftarrow \text{Gen}(1^\lambda)$, $(crs, td) \leftarrow \mathcal{S}_{crs}(1^\lambda)$;
- Run $\mathcal{A}(pk_0, pk_1, crs)$, answer $\mathcal{O}_{Dec}$ using sk_0
- On (m_0, m_1) : forward to $\text{Chal}_\Pi^{\text{IND-CPA}}$, receive $c_1^* = \text{Enc}(pk_1, m_b)$;
- Set $c_0^* \leftarrow \text{Enc}(pk_0, m_0)$, $\pi^* \leftarrow \mathcal{S}_{prf}(crs, td, (pk_0, pk_1, c_0^*, c_1^*))$
- Give $ct^* = (c_0^*, c_1^*, \pi^*)$ to $\mathcal{A}$; output $\mathcal{A}$'s bit b^*

Hop $G_1 \rightarrow G_2$: IND-CPA of pk_1

Only change: c_1^* encrypts m_1 instead of m_0 . Reduction to IND-CPA.

$\mathcal{B}$ against IND-CPA of Π

- Receive pk_1 ; $(pk_0, sk_0) \leftarrow \text{Gen}(1^\lambda)$, $(crs, td) \leftarrow \mathcal{S}_{crs}(1^\lambda)$;
- Run $\mathcal{A}(pk_0, pk_1, crs)$, answer $\mathcal{O}_{Dec}$ using sk_0
- On (m_0, m_1) : forward to $\text{Chal}_{\Pi}^{\text{IND-CPA}}$, receive $c_1^* = \text{Enc}(pk_1, m_b)$;
- Set $c_0^* \leftarrow \text{Enc}(pk_0, m_0)$, $\pi^* \leftarrow \mathcal{S}_{prf}(crs, td, (pk_0, pk_1, c_0^*, c_1^*))$
- Give $ct^* = (c_0^*, c_1^*, \pi^*)$ to $\mathcal{A}$; output $\mathcal{A}$'s bit b^*

- $\mathcal{B}$ knows neither sk_1 nor b – not needed: $\mathcal{O}_{Dec}$ uses sk_0 and c_0^* encrypts m_0 in both games; π^* needs no witness.
- $b = 0$: $\mathcal{A}$ sees G_1 ; $b = 1$: $\mathcal{A}$ sees G_2 . $\Rightarrow \epsilon_2 = \text{Adv}_{\Pi, \mathcal{B}}^{\text{IND-CPA}}(\lambda)$

Proof of Theorem 3.2: where are we?

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge
G_2	m_0	m_1	simulated	sk_0	IND-CPA of pk_1
G_3	m_0	m_1	simulated	sk_1	soundness
G_4	m_1	m_1	simulated	sk_1	IND-CPA of pk_0
G_5	m_1	m_1	simulated	sk_0	soundness
G_6	m_1	m_1	real	sk_0	zero-knowledge

Hop $G_2 \rightarrow G_3$: switching the decryption key

Only change: $\mathcal{O}_{\text{Dec}}$ answers $\text{Dec}(\textcolor{violet}{sk}_1, c_1)$ instead of $\text{Dec}(\textcolor{violet}{sk}_0, c_0)$ (still after checking π).

Hop $G_2 \rightarrow G_3$: switching the decryption key

Only change: $\mathcal{O}_{\text{Dec}}$ answers $\text{Dec}(\textcolor{violet}{sk}_1, c_1)$ instead of $\text{Dec}(\textcolor{violet}{sk}_0, c_0)$ (still after checking π).

Define the event Bad: $\mathcal{A}$ makes a $\mathcal{O}_{\text{Dec}}$ query (c_0, c_1, π) with

$$\text{Vfy}(\text{crs}, (\textcolor{blue}{pk}_0, \textcolor{blue}{pk}_1, c_0, c_1), \pi) = 1 \quad \wedge \quad \text{Dec}(\textcolor{violet}{sk}_0, c_0) \neq \text{Dec}(\textcolor{violet}{sk}_1, c_1).$$

Hop $G_2 \rightarrow G_3$: switching the decryption key

Only change: $\mathcal{O}_{\text{Dec}}$ answers $\text{Dec}(\textcolor{violet}{sk}_1, c_1)$ instead of $\text{Dec}(\textcolor{violet}{sk}_0, c_0)$ (still after checking π).

Define the event Bad: $\mathcal{A}$ makes a $\mathcal{O}_{\text{Dec}}$ query (c_0, c_1, π) with

$$\text{Vfy}(\text{crs}, (\textcolor{blue}{pk}_0, \textcolor{blue}{pk}_1, c_0, c_1), \pi) = 1 \quad \wedge \quad \text{Dec}(\textcolor{violet}{sk}_0, c_0) \neq \text{Dec}(\textcolor{violet}{sk}_1, c_1).$$

- Unless Bad, every oracle answer is identical in G_2 and $G_3 \Rightarrow$ identical view of $\mathcal{A}$. Hence: $\epsilon_3 \leq \Pr[\text{Bad in } G_2]$.

Hop $G_2 \rightarrow G_3$: switching the decryption key

Only change: $\mathcal{O}_{\text{Dec}}$ answers $\text{Dec}(\textcolor{violet}{sk}_1, c_1)$ instead of $\text{Dec}(\textcolor{violet}{sk}_0, c_0)$ (still after checking π).

Define the event Bad: $\mathcal{A}$ makes a $\mathcal{O}_{\text{Dec}}$ query (c_0, c_1, π) with

$$\text{Vfy}(\text{crs}, (\textcolor{blue}{pk}_0, \textcolor{blue}{pk}_1, c_0, c_1), \pi) = 1 \quad \wedge \quad \text{Dec}(\textcolor{violet}{sk}_0, c_0) \neq \text{Dec}(\textcolor{violet}{sk}_1, c_1).$$

- Unless Bad, every oracle answer is identical in G_2 and $G_3 \Rightarrow$ identical view of $\mathcal{A}$. Hence: $\epsilon_3 \leq \Pr[\text{Bad in } G_2]$.
- BAD happens $\Rightarrow \mathcal{A}$ faked a proof (by correctness of Π)

Hop $G_2 \rightarrow G_3$: switching the decryption key

Only change: $\mathcal{O}_{\text{Dec}}$ answers $\text{Dec}(\textcolor{violet}{sk}_1, c_1)$ instead of $\text{Dec}(\textcolor{violet}{sk}_0, c_0)$ (still after checking π).

Define the event Bad: $\mathcal{A}$ makes a $\mathcal{O}_{\text{Dec}}$ query (c_0, c_1, π) with

$$\text{Vfy}(\text{crs}, (\textcolor{blue}{pk}_0, \textcolor{blue}{pk}_1, c_0, c_1), \pi) = 1 \quad \wedge \quad \text{Dec}(\textcolor{violet}{sk}_0, c_0) \neq \text{Dec}(\textcolor{violet}{sk}_1, c_1).$$

- Unless Bad, every oracle answer is identical in G_2 and $G_3 \Rightarrow$ identical view of $\mathcal{A}$. Hence: $\epsilon_3 \leq \Pr[\text{Bad in } G_2]$.
- BAD happens $\Rightarrow \mathcal{A}$ faked a proof (by correctness of Π)
- Intuitively, Bad violates soundness. *BUT we need to be careful!*

Hop $G_2 \rightarrow G_3$: switching the decryption key

- Goal: show that $\epsilon_3 \leq \Pr[\text{Bad in } G_2]$
- Approach: reduce to soundness of NIZK

Hop $G_2 \rightarrow G_3$: switching the decryption key

- Goal: show that $\epsilon_3 \leq \Pr[\text{Bad in } G_2]$
- Approach: reduce to soundness of NIZK
- Recall that in G_2 :
 - crs is *simulated*

Hop $G_2 \rightarrow G_3$: switching the decryption key

- Goal: show that $\epsilon_3 \leq \Pr[\text{Bad in } G_2]$
- Approach: reduce to soundness of NIZK
- Recall that in G_2 :
 - crs is *simulated*
- Recall that in the soundness game
 - crs is *honestly generated*!

Hop $G_2 \rightarrow G_3$: switching the decryption key

- Goal: show that $\epsilon_3 \leq \Pr[\text{Bad in } G_2]$
- Approach: reduce to soundness of NIZK
- Recall that in G_2 :
 - crs is *simulated*
- Recall that in the soundness game
 - crs is *honestly generated*!
- We can't reduce directly!

Lemma 3.4: Consistency under a simulated crs

There are PPT $\mathcal{B}_1, \mathcal{D}'$ such that

$$\Pr[\text{Bad}] \leq \text{Adv}_{\text{NIZK}, \mathcal{B}_1}^{\text{sound}}(\lambda) + \text{Adv}_{\text{NIZK}, \mathcal{D}'}^{\text{ZK}}(\lambda).$$

Proof of Lemma 3.4 : honest crs

Let G'_2 be G_2 with an honest $crs \leftarrow \text{Setup}(1^\lambda)$. (Only the query phase matters for Bad.)

$\mathcal{B}_1$ against soundness

- receive crs ; generate $(pk_0, sk_0), (pk_1, sk_1)$; run $\mathcal{A}$'s query phase, answering $\mathcal{O}_{\text{Dec}}$ with sk_0
- if some query (c_0, c_1, π) has $\text{Vfy} = 1$ and $\text{Dec}(sk_0, c_0) \neq \text{Dec}(sk_1, c_1)$:
output $x := (pk_0, pk_1, c_0, c_1)$ and π

Proof of Lemma 3.4 : honest crs

Let G'_2 be G_2 with an honest $crs \leftarrow \text{Setup}(1^\lambda)$. (Only the query phase matters for Bad.)

$\mathcal{B}_1$ against soundness

- receive crs ; generate $(pk_0, sk_0), (pk_1, sk_1)$; run $\mathcal{A}$'s query phase, answering $\mathcal{O}_{\text{Dec}}$ with sk_0
 - if some query (c_0, c_1, π) has $\text{Vfy} = 1$ and $\text{Dec}(sk_0, c_0) \neq \text{Dec}(sk_1, c_1)$:
output $x := (pk_0, pk_1, c_0, c_1)$ and π
- $\mathcal{B}_1$ simulates G'_2 perfectly (it knows both keys).
 - $\text{Dec}(sk_0, c_0) \neq \text{Dec}(sk_1, c_1) \Rightarrow x \notin \mathcal{L}_{\text{NY}}$ (if $x \in \mathcal{L}_{\text{NY}}$, correctness gives the same m on both sides).

Proof of Lemma 3.4 : honest crs

Let G'_2 be G_2 with an honest $crs \leftarrow \text{Setup}(1^\lambda)$. (Only the query phase matters for Bad.)

$\mathcal{B}_1$ against soundness

- receive crs ; generate $(pk_0, sk_0), (pk_1, sk_1)$; run $\mathcal{A}$'s query phase, answering $\mathcal{O}_{\text{Dec}}$ with sk_0
 - if some query (c_0, c_1, π) has $\text{Vfy} = 1$ and $\text{Dec}(sk_0, c_0) \neq \text{Dec}(sk_1, c_1)$:
output $x := (pk_0, pk_1, c_0, c_1)$ and π
- $\mathcal{B}_1$ simulates G'_2 perfectly (it knows both keys).
 - $\text{Dec}(sk_0, c_0) \neq \text{Dec}(sk_1, c_1) \Rightarrow x \notin \mathcal{L}_{\text{NY}}$ (if $x \in \mathcal{L}_{\text{NY}}$, correctness gives the same m on both sides).

$$\Pr[\text{Bad in } G'_2] \leq \text{Adv}_{\text{NIZK}, \mathcal{B}_1}^{\text{sound}}(\lambda).$$

Proof of Lemma 3.4: simulated vs. honest *crs*

$\mathcal{D}' = (\mathcal{D}'_0, \mathcal{D}'_1)$ against ZK

$\mathcal{D}'_0(crs)$: generate both key pairs; run $\mathcal{A}$'s query phase, answering $\mathcal{O}_{Dec}$ with sk_0 ; set $\beta := 1$ iff Bad occurred; output some true $(x, w) \in \mathcal{R}_{NY}$ and $st := \beta$

$\mathcal{D}'_1(st, \pi)$: ignore π , output β

Proof of Lemma 3.4: simulated vs. honest *crs*

$\mathcal{D}' = (\mathcal{D}'_0, \mathcal{D}'_1)$ against ZK

$\mathcal{D}'_0(crs)$: generate both key pairs; run $\mathcal{A}$'s query phase, answering $\mathcal{O}_{Dec}$ with sk_0 ; set $\beta := 1$ iff Bad occurred; output some true $(x, w) \in \mathcal{R}_{NY}$ and $st := \beta$
 $\mathcal{D}'_1(st, \pi)$: ignore π , output β

- $\text{ExpReal}^{\text{ZK}}$: $\mathcal{A}$ runs in G'_2 . $\text{ExpIdeal}^{\text{ZK}}$: $\mathcal{A}$ runs in G_2 .
- Hence $|\Pr[\text{Bad in } G_2] - \Pr[\text{Bad in } G'_2]| = \text{Adv}_{\text{NIZK}, \mathcal{D}'}^{\text{ZK}}(\lambda)$; with Step 1 the Lemma 3.4 follows. □

Proof of Lemma 3.4: simulated vs. honest *crs*

$\mathcal{D}' = (\mathcal{D}'_0, \mathcal{D}'_1)$ against ZK

$\mathcal{D}'_0(crs)$: generate both key pairs; run $\mathcal{A}$'s query phase, answering $\mathcal{O}_{Dec}$ with sk_0 ; set $\beta := 1$ iff Bad occurred; output some true $(x, w) \in \mathcal{R}_{NY}$ and $st := \beta$
 $\mathcal{D}'_1(st, \pi)$: ignore π , output β

- $\text{ExpReal}^{\text{ZK}}$: $\mathcal{A}$ runs in G'_2 . $\text{ExpIdeal}^{\text{ZK}}$: $\mathcal{A}$ runs in G_2 .
- Hence $|\Pr[\text{Bad in } G_2] - \Pr[\text{Bad in } G'_2]| = \text{Adv}_{\text{NIZK}, \mathcal{D}'}^{\text{ZK}}(\lambda)$; with Step 1 the Lemma 3.4 follows. □

$\mathcal{D}'$ never has to give $\mathcal{A}$ a proof: all queries come *before* ct^* .

Proof of Theorem 3.2: where are we?

game	c_0^* encrypts	c_1^* encrypts	crs, π^*	$\mathcal{O}_{Dec}$ key	$G_{i-1} \rightarrow G_i$ via
G_0	m_0	m_0	real	sk_0	—
G_1	m_0	m_0	simulated	sk_0	zero-knowledge
G_2	m_0	m_1	simulated	sk_0	IND-CPA of pk_1
G_3	m_0	m_1	simulated	sk_1	soundness
G_4	m_1	m_1	simulated	sk_1	IND-CPA of pk_0
G_5	m_1	m_1	simulated	sk_0	soundness
G_6	m_1	m_1	real	sk_0	zero-knowledge

Hops $G_3 \rightarrow G_4 \rightarrow G_5 \rightarrow G_6$: by symmetry

- $G_3 \rightarrow G_4$: c_0^* encrypts m_1 instead of m_0 . Reduction $\mathcal{B}'$ to IND-CPA of pk_0 ; $\mathcal{B}'$ generates (pk_1, sk_1) itself and answers $\mathcal{O}_{\text{Dec}}$ with sk_1 — *this is why we switched keys*. $\epsilon_4 = \text{Adv}_{\Pi, \mathcal{B}'}^{\text{IND-CPA}}$.

Hops $G_3 \rightarrow G_4 \rightarrow G_5 \rightarrow G_6$: by symmetry

- $G_3 \rightarrow G_4$: c_0^* encrypts m_1 instead of m_0 . Reduction $\mathcal{B}'$ to IND-CPA of pk_0 ; $\mathcal{B}'$ generates (pk_1, sk_1) itself and answers $\mathcal{O}_{Dec}$ with sk_1 — *this is why we switched keys*. $\epsilon_4 = \text{Adv}_{\Pi, \mathcal{B}'}^{\text{IND-CPA}}$.
- $G_4 \rightarrow G_5$: switch $\mathcal{O}_{Dec}$ back to sk_0 . Same Bad event, same Lemma 3.4: $\epsilon_5 \leq \text{Adv}_{\text{NIZK}, \mathcal{B}_1'}^{\text{sound}} + \text{Adv}_{\text{NIZK}, \mathcal{D}''}^{\text{ZK}}$.

Hops $G_3 \rightarrow G_4 \rightarrow G_5 \rightarrow G_6$: by symmetry

- $G_3 \rightarrow G_4$: c_0^* encrypts m_1 instead of m_0 . Reduction $\mathcal{B}'$ to IND-CPA of pk_0 ; $\mathcal{B}'$ generates (pk_1, sk_1) itself and answers $\mathcal{O}_{Dec}$ with sk_1 — *this is why we switched keys*. $\epsilon_4 = \text{Adv}_{\Pi, \mathcal{B}'}^{\text{IND-CPA}}$.
- $G_4 \rightarrow G_5$: switch $\mathcal{O}_{Dec}$ back to sk_0 . Same Bad event, same Lemma 3.4: $\epsilon_5 \leq \text{Adv}_{\text{NIZK}, \mathcal{B}'_1}^{\text{sound}} + \text{Adv}_{\text{NIZK}, \mathcal{D}''}^{\text{ZK}}$.
- $G_5 \rightarrow G_6$: real crs , real π^* . Both c_i^* encrypt m_1 , so a witness exists. $\epsilon_6 = \text{Adv}_{\text{NIZK}, \mathcal{D}'''}^{\text{ZK}}$.

Hops $G_3 \rightarrow G_4 \rightarrow G_5 \rightarrow G_6$: by symmetry

- $G_3 \rightarrow G_4$: c_0^* encrypts m_1 instead of m_0 . Reduction $\mathcal{B}'$ to IND-CPA of pk_0 ; $\mathcal{B}'$ generates (pk_1, sk_1) itself and answers $\mathcal{O}_{Dec}$ with sk_1 — *this is why we switched keys*. $\epsilon_4 = \text{Adv}_{\Pi, \mathcal{B}'}^{\text{IND-CPA}}$.
- $G_4 \rightarrow G_5$: switch $\mathcal{O}_{Dec}$ back to sk_0 . Same Bad event, same Lemma 3.4: $\epsilon_5 \leq \text{Adv}_{\text{NIZK}, \mathcal{B}'_1}^{\text{sound}} + \text{Adv}_{\text{NIZK}, \mathcal{D}''}^{\text{ZK}}$.
- $G_5 \rightarrow G_6$: real crs , real π^* . Both c_i^* encrypt m_1 , so a witness exists. $\epsilon_6 = \text{Adv}_{\text{NIZK}, \mathcal{D}'''}^{\text{ZK}}$.

G_6 is exactly the IND-CCA1 experiment with $b = 1$.

Proof of Theorem 3.2: summing up

$$\text{Adv}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(\lambda) \leq \sum_{i=1}^6 \epsilon_i \leq 2 \text{Adv}_{\Pi, \cdot}^{\text{IND-CPA}} + 2 \text{Adv}_{\text{NIZK}, \cdot}^{\text{sound}} + 4 \text{Adv}_{\text{NIZK}, \cdot}^{\text{ZK}}.$$

(all adversaries we built are PPT: time of $\mathcal{A}$ plus q_D decryptions)

Proof of Theorem 3.2: summing up

$$\text{Adv}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(\lambda) \leq \sum_{i=1}^6 \epsilon_i \leq 2 \text{Adv}_{\Pi, \cdot}^{\text{IND-CPA}} + 2 \text{Adv}_{\text{NIZK}, \cdot}^{\text{sound}} + 4 \text{Adv}_{\text{NIZK}, \cdot}^{\text{ZK}}.$$

(all adversaries we built are PPT: time of $\mathcal{A}$ plus q_D decryptions)

- Each term is negligible by assumption $\Rightarrow \Pi_{\text{NY}}$ is IND-CCA1 secure. □
- Loss: a small constant — the reduction is *tight* up to that constant.

Proof of Theorem 3.2: summing up

$$\text{Adv}_{\Pi_{\text{NY}}, \mathcal{A}}^{\text{IND-CCA1}}(\lambda) \leq \sum_{i=1}^6 \epsilon_i \leq 2 \text{Adv}_{\Pi, \cdot}^{\text{IND-CPA}} + 2 \text{Adv}_{\text{NIZK}, \cdot}^{\text{sound}} + 4 \text{Adv}_{\text{NIZK}, \cdot}^{\text{ZK}}.$$

(all adversaries we built are PPT: time of $\mathcal{A}$ plus q_D decryptions)

- Each term is negligible by assumption $\Rightarrow \Pi_{\text{NY}}$ is IND-CCA1 secure. □
- Loss: a small constant — the reduction is *tight* up to that constant.

In every game, whoever answers $\mathcal{O}_{\text{Dec}}$ *knows* the key it uses; soundness guarantees that *both* keys give the same answer.

Where exactly did we use IND-CCA1?

- Only in Lemma 3.4 (Step 2)
the ZK distinguisher $\mathcal{D}'$ ran $\mathcal{A}$'s *query phase* without ever giving $\mathcal{A}$ a proof.
- IND-CCA2: $\mathcal{A}$ also queries $\mathcal{O}_{\text{Dec}}$ *after* seeing $ct^* = (c_0^*, c_1^*, \pi^*)$
- In G_2, G_3, G_4 :
 - π^* is a **simulated proof of a false statement** (c_0^*, c_1^* encrypt different messages),
 - and we need: $\mathcal{A}$ still cannot produce a valid proof of *another* false statement.

Where exactly did we use IND-CCA1?

- Only in Lemma 3.4 (Step 2)
the ZK distinguisher $\mathcal{D}'$ ran $\mathcal{A}$'s *query phase* without ever giving $\mathcal{A}$ a proof.
- IND-CCA2: $\mathcal{A}$ also queries $\mathcal{O}_{\text{Dec}}$ *after* seeing $ct^* = (c_0^*, c_1^*, \pi^*)$
- In G_2, G_3, G_4 :
 - π^* is a **simulated proof of a false statement** (c_0^*, c_1^* encrypt different messages),
 - and we need: $\mathcal{A}$ still cannot produce a valid proof of *another* false statement.

Where exactly did we use IND-CCA1?

- Only in Lemma 3.4 (Step 2)
the ZK distinguisher $\mathcal{D}'$ ran $\mathcal{A}$'s *query phase* without ever giving $\mathcal{A}$ a proof.
- IND-CCA2: $\mathcal{A}$ also queries $\mathcal{O}_{\text{Dec}}$ *after* seeing $ct^* = (c_0^*, c_1^*, \pi^*)$
- In G_2, G_3, G_4 :
 - π^* is a **simulated proof of a false statement** (c_0^*, c_1^* encrypt different messages),
 - and we need: $\mathcal{A}$ still cannot produce a valid proof of *another* false statement.

$\Rightarrow$ IND-CCA2 needs a NIZK whose soundness **survives seeing a simulated proof.**